Criterion C: System overview

System models

UML diagrams of system models depend on the computational context of the solution. It is important that students choose the appropriate diagram style for their chosen computational context. They should also be encouraged to use the appropriate number of diagrams, as additional unnecessary diagrams can detract from the solution.

The system model, along with its individual components in the system overview, should be sufficiently clear that a third party with the necessary programming knowledge would be able to complete the solution. The system model on its own is a higher-level view of the system showing how the individual components connect with each other. The model will be related to the decomposition of the problem, as each component will represent a solution of the sub-problems in the decomposed problem. There are multiple tools to facilitate this.

- For procedural coding, a flow-charting tool can break down the modules from the planning stage.
- For procedural coding, a bulleted list with indentations might be used.
- For object-oriented coding, it is important to document the component objects using UML diagrams that show dependencies between objects.
- A database project would require table design with proper normalization and a description of data dictionaries.

Annotated design sketches of the UI should be part of this stage of the design process.

Algorithms as components in a system model

The individual components in a system model are made up of algorithms. Each of these algorithms should be clearly seen individually in section C of the computational solution and relate directly to the components in the system model.

The design of specific algorithms can be shown as:

- pseudocode

- flow charts
- bulleted lists with indentations.

Testing strategies

At this point in the planning stage, students need to consider testing strategies. A variety of rigorous tests need to be developed to assess the functionality of the product against the success criteria as stated in the problem specification (criterion A).

In addition to functional testing, the product needs to be tested structurally using a variety of valid, extreme and invalid data that covers all possible situations.

A testing strategy for functional testing could be fulfilled with a table that lists the success criteria, and a separate column with descriptions of how each separate success criterion will be tested.

A testing strategy for structural testing should focus on the major algorithms and whether they perform correctly. This could be fulfilled by adding tables that specify valid, extreme and invalid input data, together with a column for the expected results.

Comprehensive structural testing also requires students to address all eventualities. For example, when adding to a sorted list, data needs to be added to the empty list, to the end of the list, to the start of the list and in the middle of the list. The following data would structurally test the add method for a sorted list of animals: horse, zebra, donkey, mule.